LLM 为什么会「幻觉」?

从数学原理到工程业务的两层理解

2026 年 4 月 21 日

目录

1 这一节到底要解释什么 **3**

2 讲解原则 **3**

3 零基础最该问的 6 个问题 **3**

4 Q1：LLM 在算什么？——下一个 token 的条件分布 **4**

4.1 背景问题 4

4.2 朴素直觉 4

4.3 数学表达 4

4.4 工程对应 5

5 Q2：训练目标——为什么它学会说话却没学会说真话 **5**

5.1 背景问题 5

5.2 朴素直觉 5

5.3 数学表达：交叉熵 = 极大似然 5

5.4 工程对应 6

6 Q3：为什么模型加大也救不了 **6**

6.1 有限近似 + 长尾事实 6

6.2 有限模型容量带来的「最近邻幻觉」 6

6.3 工程对应 6

7 Q4：采样参数怎么放大或抑制幻觉 **7**

7.1 贪心、温度、top-k、top-p 简介 7

7.2 对幻觉的影响 7

7.3 工程对应 7

8 Q5：模型「知不知道」自己在编？——置信度与校准 **7**

8.1 什么叫「校准」 7

8.2 数学表达 8

8.3 工程对应 8

9 Q6：工程上把幻觉压下去的几条主线 **8**
9.1 主线 1：检索增强（RAG） 8
9.2 主线 2：工具调用（function calling） 9
9.3 主线 3：约束解码 / 结构化输出 9
9.4 主线 4：后训练对齐（SFT、RLHF、DPO） 9
9.5 主线 5：评测与守门员 9

10 把整篇压成一张表 **10**

11 这一节真正该记住的 **10**

1 这一节到底要解释什么

问题

你在和大模型聊天时，可能遇到过这样的事：

- 它一本正经地给你编了一篇不存在的论文；
- 它编出一段看上去合规但实际上对不上数据库的 SQL；
- 它在医学/法律问题上自信地说错；
- 它引用一个根本不存在的 API、URL、引文。

这些被统称为 LLM 的「**幻觉**」(**hallucination**)。
更让人困惑的是：模型并不知道自己在编。它不是「主动撒谎」，而是按某种内在机制算出来的最可能的下一句话——只是这句话恰好和现实对不上。
这一节要讲清楚：

1. **数学原理上**：训练目标本身决定了模型一定会产生幻觉，这不是 bug，是 LLM 的统计本性；
2. **工程业务上**：实际系统通过哪些手段把幻觉的影响压到可接受范围。

朴素直觉

最快的一句话总结：

LLM 不是「记住事实」，而是「拟合下一个 token 的分布」。拟合出来的分布在「事实」这个维度上没有任何特别约束，所以它有时候会给「错的事实」分配最高概率。

记住这一点

幻觉的根源不是模型「不够大」，而是训练目标从来没要求过它说真话——只要求它说统计上像训练语料的话。

2 讲解原则

每一节都按四步走：

1. **背景问题**——上一步还差什么？
2. **朴素直觉**——先用人话讲清楚。
3. **数学表达**——再写公式，每个新符号当场翻译。
4. **工程对应**——这背后是什么实际产品决策。

3 零基础最该问的 6 个问题

Q1. LLM 到底在算什么？什么叫「下一个 token 的概率分布」？

Q2. 训练目标是什么？为什么这个目标就足以让它「会说话」却不保证它说的是真话？

Q3. 为什么扩大模型/加更多数据，不能彻底消除幻觉？

Q4.「采样温度」「top-k」「top-p」这些采样参数和幻觉是什么关系？

Q5. 模型自己知不知道它在编？什么是「校准（calibration）」？

Q6. 工程上能用 RAG、工具调用、约束解码、人工反馈把幻觉压到什么程度？

下面所有节都在回答其中一个或几个问题。

4 Q1：LLM 在算什么？——下一个 token 的条件分布

4.1 背景问题

要先把术语对齐：模型「输出文字」的过程，本质上是在做概率建模。

4.2 朴素直觉

朴素直觉

把一段文本想象成一串单元（叫 **token**，可以粗略当成「子词」）。模型的工作就是：

> *已经看见了前面 $t-1$ 个 token，下一个 token 是「the」「a」「dog」……每个的概率是多少？*

注意：是概率，不是「答案」。模型给出一张覆盖全词表（几万个词）的概率表，然后采样器从这张表里挑一个，作为下一个词。

4.3 数学表达

记 token 序列为 $w_1, w_2, \ldots, w_T$，词表为 $\mathcal{V}$，模型参数为 θ。模型定义的是**条件分布**：

$$p_\theta(w_t \mid w_1, \ldots, w_{t-1}) \in \Delta^{|\mathcal{V}|-1},$$

也就是「在前文给定的条件下，下一个 token 的概率分布」。整段文本的概率被自回归地分解成

$$p_\theta(w_1, \ldots, w_T) \;=\; \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t}).$$

备注 4.1. $\Delta^{|\mathcal{V}|-1}$ 是「概率单纯形」的写法：$|\mathcal{V}|$ 个非负数，加起来等于 *1*。简单理解为「合法的概率分布」即可。

4.4 工程对应

工程视角

LLM 服务的接口几乎都长这样：

- 输入：一段 prompt；
- 输出：一串 token，伴随每一步的概率（或 logprob）。

所谓「写文章」「写代码」「答题」都是同一件事：*反复采样下一个 token*。模型并不区分「写小说」（创作）和「答事实题」（事实）这两种语境；它只是在分布上「看哪种在训练语料里更常见」。

5 Q2：训练目标——为什么它学会说话却没学会说真话

5.1 背景问题

模型用什么目标训练？这个目标和「真假」有什么关系？

5.2 朴素直觉

朴素直觉

预训练只做一件事：**对着海量文本，学会预测下一个 token**。真就是这么简单。模型的「老师」就是*下一个真实出现过的 token*。**没有人告诉它「这句话是真的」「那句话是假的」**——训练语料里有真，也有假，也有虚构小说，也有错的网页。模型学的是*平均分布*。

5.3 数学表达：交叉熵 = 极大似然

设训练数据来自真实分布 p_{data}，模型分布是 p_θ。预训练损失就是**交叉熵**：

$$\mathcal{L}(\theta) \;=\; -\,\mathbb{E}_{w\sim p_{\text{data}}}\left[\sum_{t=1}^{T}\log p_\theta(w_t \mid w_{<t})\right].$$

最小化它，等价于最小化

$$\mathrm{KL}\big(p_{\text{data}}\,\|\,p_\theta\big) \;=\; \sum_{w} p_{\text{data}}(w)\log\frac{p_{\text{data}}(w)}{p_\theta(w)}.$$

也就是说：

> 训练目标就是「让 p_θ 尽可能贴近 p_{data}」。

为什么这一定会产生幻觉

关键在于「真实分布」是文本分布，*不是事实分布*。形式上：

$$p_{\text{data}}(\text{文本}) \;\neq\; \mathbb{1}\{\text{文本是真的}\}.$$

也就是：训练分布把「合理的语言」都打上了非零概率，*包括语法通顺但事实错误的话*。模型完美拟合 p_{data} 的代价，就是它会以非零概率生成这些「合理的假话」。

5.4 工程对应

工程视角

这就是「幻觉是设计上的副产品」的真正含义。任何只用「下一个 token 似然」做监督的预训练，都无法保证「事实正确」这个外部约束。后训练（SFT、RLHF、DPO 等）只能在分布上挪一挪权重，不能从根上把「假话」的概率压到 0。

6 Q3：为什么模型加大也救不了

6.1 有限近似 + 长尾事实

朴素直觉

就算模型能完美拟合 p_{data}，幻觉也救不了，原因有两个：

1. p_{data} **自身有错**：训练语料里就有错的、过时的、自相矛盾的内容。让模型「学得越像」，它把这些错误也学得越像。
2. **长尾事实**：很多事实（比如「某小镇 2023 年的镇长是谁」）在训练语料里几乎不出现，分布在这种位置上几乎是「均匀噪声」，采样出来的几乎一定是错的。

6.2 有限模型容量带来的「最近邻幻觉」

近似误差

现实中模型容量有限，参数 θ 是近似解。对一个真实事实 $w^\star$ 周围，模型分布通常长这样：

$$p_\theta(w \mid c) \approx \sum_i \pi_i \, \delta_{w^{(i)}},$$

即在若干个「在训练语料里和 c 配对过的相似回答」上分散概率。当真实答案不在这若干个里，模型只能从最像的几个里挑——这正是「听起来对、查起来错」的典型形态。

一个粗糙的下界（直觉证明）

如果某条事实在训练语料里出现的频率是 ϵ，那么哪怕模型把全部「学习容量」都用在这一条上，它的最佳判别准确率也受限于它见过的样本数。**结论：**规模不能消除幻觉，最多让它更少、更隐蔽。

6.3 工程对应

工程视角

这就是为什么所有严肃的产品都不会「裸用」一个大模型来回答事实问题，而是必须配合**外部知识源**（数据库、搜索、文档库）。模型负责语言，外部源负责事实——这是真正的工程分工。

7 Q4：采样参数怎么放大或抑制幻觉

7.1 贪心、温度、top-k、top-p 简介

设模型给出 logits $z \in \mathbb{R}^{|\mathcal{V}|}$，经过温度缩放的 softmax 是

$$p_T(w) = \frac{\exp(z_w/T)}{\sum_{w'} \exp(z_{w'}/T)}.$$

温度 T 的几个极限：

- $T \to 0^+$：分布塌成最大概率位置——**贪心**。
- $T = 1$：原始分布。
- $T \to \infty$：分布趋向均匀——几乎纯噪声。

top-k 是只在概率最高的 k 个 token 里采样；**top-p（核采样）**是在累计概率刚好 $\geq p$ 的最小集合里采样。两者都是「砍掉长尾」。

7.2 对幻觉的影响

两种典型场景

- T **越大，幻觉越多**。高温让低概率 token 被采到的机会大幅上升，「看起来合理但对不上事实」的回答更频繁。
- $T = 0$ / **贪心也未必安全**。如果模型在某个事实位置上的最大概率本身就指向「错答案」，贪心会每次都吐出同一个错答案（更难被察觉）。

7.3 工程对应

工程视角

实际服务通常按任务选采样：

- **创意写作**：高温、宽 top-p，鼓励多样；
- **代码 / 事实问答 / SQL**：低温、窄 top-p，甚至贪心；
- **多次采样 + 投票（self-consistency）**：同一个问题采若干次，挑「答案出现最多」的那一个，在数学题、程序题上能明显降低单次幻觉。

8 Q5：模型「知不知道」自己在编？——置信度与校准

8.1 什么叫「校准」

朴素直觉

一个模型说「我有 90% 把握」时，如果它「真的」在这种情况里 90% 的时候是对的，我们就说它**校准（calibrated）**得好。**幻觉的麻烦不只是「错」，更是「错得很自信」**。

8.2 数学表达

校准误差

设模型对答案 $\hat{y}$ 的自报置信度是 $\hat{p}$，真实正确率是 $\Pr(\hat{y} = y \mid \hat{p})$。完美校准要求

$$\Pr(\hat{y} = y \mid \hat{p} = q) \;=\; q, \qquad \forall q \in [0, 1].$$

校准误差用 **Expected Calibration Error (ECE)** 衡量：

$$\mathrm{ECE} \;=\; \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

其中 B_m 是按置信度分桶后的样本，$\mathrm{acc}, \mathrm{conf}$ 分别是该桶的真实正确率和平均自报置信度。

为什么 RLHF 之后校准反而变差

很多研究观察到：仅用预训练时模型的 token 概率 相对校准；经过 RLHF 之后，模型学会「显得自信」，对开放问答自信地说错的频率反而更高。这是因为 RLHF 的奖励信号偏好「果断、流畅」的回答，不偏好「我不确定」。

8.3 工程对应

工程视角

工程上常见的几种「让模型坦白不确定」的手段：

- 在 system prompt 里硬性要求：「不知道就说不知道」；
- 用模型自报置信度（*verbalized confidence*）做下游过滤；
- 使用模型 logprob 做 token 级不确定度估计；
- 对长答案做自评分（*self-critique*）或多模型互评；
- 关键场景接**弃答**逻辑：低于阈值就回退到「请提供更多信息」。

9 Q6：工程上把幻觉压下去的几条主线

9.1 主线 1：检索增强（RAG）

朴素直觉

不让模型「凭记忆」答事实题，而是先检索相关文档片段，再让模型「基于这段材料」回答。模型的角色从「百科全书」变成「会读书的助手」。

形式化

设外部知识库 $\mathcal{D}$，给定问题 q，先用检索器 R 取出片段 $D_q = R(q, \mathcal{D})$，然后采样

$$\hat{a} \sim p_\theta(\cdot \mid q, D_q).$$

这把答案的事实成分「锚定」到 D_q 上。条件越强，幻觉空间越小。

常见失败模式：检索召回不到正确片段、模型忽视片段（「context drift」）、片段彼此矛盾。所以 RAG 不是银弹，而是一条非常有效的减幻觉路径。

9.2 主线 2：工具调用（function calling）

工程视角

对「能算就别猜」的事实（数学、SQL、日期、汇率、代码运行结果）：让模型调用工具，把答案算出来再回写。计算器、SQL 引擎、单元测试，全是工具。模型负责决定调什么、传什么参数，工具负责给出真值。

9.3 主线 3：约束解码 / 结构化输出

把输出空间从「全词表」收窄到「合法语法」：

- JSON Schema 约束 → 输出一定是合法 JSON；
- Regex / CFG 约束 → 输出符合特定语法；
- 选项受限 → 答案必须从给定候选里选。

这本质上是在采样时把不合法 *token* 的概率直接置零，形式化是「条件分布在受限子集上的归一化」。对「编出格式错乱」「编不存在的字段名」这类幻觉特别有效。

9.4 主线 4：后训练对齐（SFT、RLHF、DPO）

让模型「更愿意说实话、更愿意承认不知道」。但要清楚：对齐改变的是分布的形状，不会让分布的支撑集（support）从「假话也有非零概率」变成「假话概率 = 0」。

9.5 主线 5：评测与守门员

工程视角

任何严肃产品都需要一个外置的**守门系统**：

- 自动评测集（针对常见事实题、易幻觉题）；
- 在线检测（关键字段做事后核查、与权威数据库比对）；
- 用户反馈环（标注 → 进 SFT/DPO 数据）；
- 业务降级策略（幻觉风险 vs. 给不出答案的成本权衡）。

10 把整篇压成一张表

幻觉的来源	数学/统计层面	工程对策
训练目标只看似然	$\min \mathrm{KL}(p_{\mathrm{data}} \| p_\theta)$，真假不在目标里	后训练对齐、奖励模型加事实奖励
语料本身有错/有虚构	p_{data} 不是事实指示函数	数据清洗、来源加权、引用约束
长尾事实样本太少	频率 ϵ 极小，估计方差大	RAG、工具调用、外部知识源
高温 / 宽 top-p 采样	长尾 token 概率被放大	任务自适应采样参数；自一致性投票
RLHF 倾向「显得自信」	校准变差，ECE 上升	verbalized confidence、弃答阈值
输出格式自由	支撑集太大	JSON / 语法约束解码
模型直接答事实题	事实信息没被「锚定」	RAG / 工具调用把事实从模型里搬出去

11 这一节真正该记住的

1. LLM 输出的是「下一个 token 的条件分布」，不是「答案」。
2. 训练目标是 *交叉熵 / 极大似然*，等价于让 p_θ 贴近 p_{data}；这一目标里*不存在*「事实正确」这个约束。
3. 因此幻觉不是 bug，而是「拟合文本分布」的必然副产品。扩大模型只能减少幻觉、不能消除幻觉。
4. 采样温度、top-k、top-p 决定了「长尾 token 被采到的概率」——这是幻觉率的一个直接旋钮。
5. 「错得很自信」是 RLHF 阶段常见的副作用，体现在校准误差（ECE）变大。
6. 工程上抑制幻觉的几条主线：*RAG*、*工具调用*、*约束解码*、*对齐训练*、*评测与守门*。没有一条是银弹，组合使用才有用。

记住这一点

一句话总结：
模型只学会了「像人话」，没学会「是真的」。
所以工程上，要把「事实」这件事从模型里*搬出去*（RAG / 工具 / 守门），
而不是寄希望于模型「再大一点就不会编了」。